

## 1 Revision History

| Date of Revision | Contributor       | Email                       |
|------------------|-------------------|-----------------------------|
| May 2023         | Chang Gao         | chang.gao@tudelft.nl        |
| May 2024         | Nicolas Chauvaux  | n.chauvaux@tudelft.nl       |
|                  | Douwe den Blanken | d.m.j.denblanken@tudelft.nl |
| Jan 2025         | Ang Li            | ang.li@tudelft.nl           |
|                  | Yizhuo Wu         | yizhuo.wu@tudelft.nl        |
| Jan 2026         | Nicolas Chauvaux  | n.chauvaux@tudelft.nl       |
|                  | Douwe den Blanken | d.m.j.denblanken@tudelft.nl |
|                  | Guilherme Guedes  | g.guedes@tudelft.nl         |

## 2 Introduction

In this laboratory exercise, you will gain a comprehensive understanding of the following topics:

- Physical implementation of synthesized circuits in **Cadence Innovus**.
- Post-Layout Simulation.

For questions on the labs and project, we'll soon open a Q/A forum on Brightspace.

You should submit **only one** report after Lab 3 to answer the questions in all labs. Detailed guidelines for the report and project will be provided after lab 3.

## 3 Tutorial: Physical Design in Cadence Innovus

Today's tutorial will provide a comprehensive walkthrough of the physical implementation process using **Cadence Innovus**. Some stages, particularly the 'route' step, can take significant time to complete. We've created Tool Command Language (**TCL**) scripts for each step to assist with this. While these steps are being executed on the server, we encourage you to familiarize yourself with the foundational concepts and **Innovus** commands we'll cover in the tutorial text.

### 3.1 Preparation

First of all, copy the files of Lab 3 and navigate to the physical design tutorial folder:

```
1 cd ~
2 cp -r /data/labs/2026/labs/lab3 ~/
3 cd lab3
4 source setup.sh
5 cd tut/pnr
```

Then, create some empty directories where the tool can write the design results.

```
1 mkdir initialReports timingReports verifyReports clockReports powerReports \
   densityReports finalReports extLogDir
```

Next, as we will use the activity from the structural simulation to optimize for power, we firstly run the synthesis:

```
1 cd ../synth
2 source run_synth.sh
```

and then the structural simulation:

```
1 # Generate firmware
2 cd ../firmware
3 make
4 # Run structural simulation
5 cd ../sim_struct
6 source run_struct_sim_vcd.sh
7 cd ../pnr
```

Before we start to use Innovus, we must provide the Process Design Kit (**PDK**) files to the tool<sup>1</sup>:

- **Standard Cell Library:** A standard cell library file (**.lib**) is a collection of pre-designed and pre-verified digital logic components, such as logic gates (AND, OR, XOR, etc.), flip-flops, and memory elements. Each cell in the library is fully characterized by the foundry, meaning its performance (timing, power, area) is accurately known for various operating conditions. This characterization is stored in .lib files (also known as Liberty files), which include information about cell functionality, timing, power, and more. Synthesis and timing analysis tools use the **.lib** files to optimize the design and check whether it meets the required timing and power constraints.
- **Technology LEF:** The Library Exchange Format (**LEF**) is a standard for representing the physical characteristics of a semiconductor design. The Technology LEF file, specifically, provides details about the manufacturing process layers and their properties, such as metal layers, via definitions and design rule information.
- **Cell LEF:** This type of LEF file provides an abstract view of the standard cells in the library. It contains information about the physical layout of these cells, but unlike **.lib** files, which include detailed functional and timing information, Cell LEF files focus solely on the physical characteristics. These characteristics include cell dimensions, abstract shapes of its layers, and pin locations. However, it's important to note that the Cell LEF does not present a fully

<sup>1</sup>To know more about these files: <https://teamvlsi.com/2020/05/lef-lef-file-in-asic-design.html>

detailed layout view but rather an abstract representation suitable for higher-level physical design and analysis.

- **Quantus RC Extraction Technology (QRC):** A QRC file (`.tch` in our case) contains information about the parasitic extraction, providing the accurate data required for post-layout simulation and timing analysis.

Start Innovus with the following command:

```
1 innovus
```

Once the Graphical User Interface (**GUI**) is shown on your screen, switch to the command-line window (on your terminal) and run the following commands line by line to set paths to the aforementioned files and load our synthesized design:

```
1 source ./scripts/1.set_variable.tcl
2 source ./scripts/2.0.load_design.tcl
3 source ./scripts/2.1.set_library_n_sdc.tcl
```

Then your layout should show a grid defining the die area as in Figure 1.

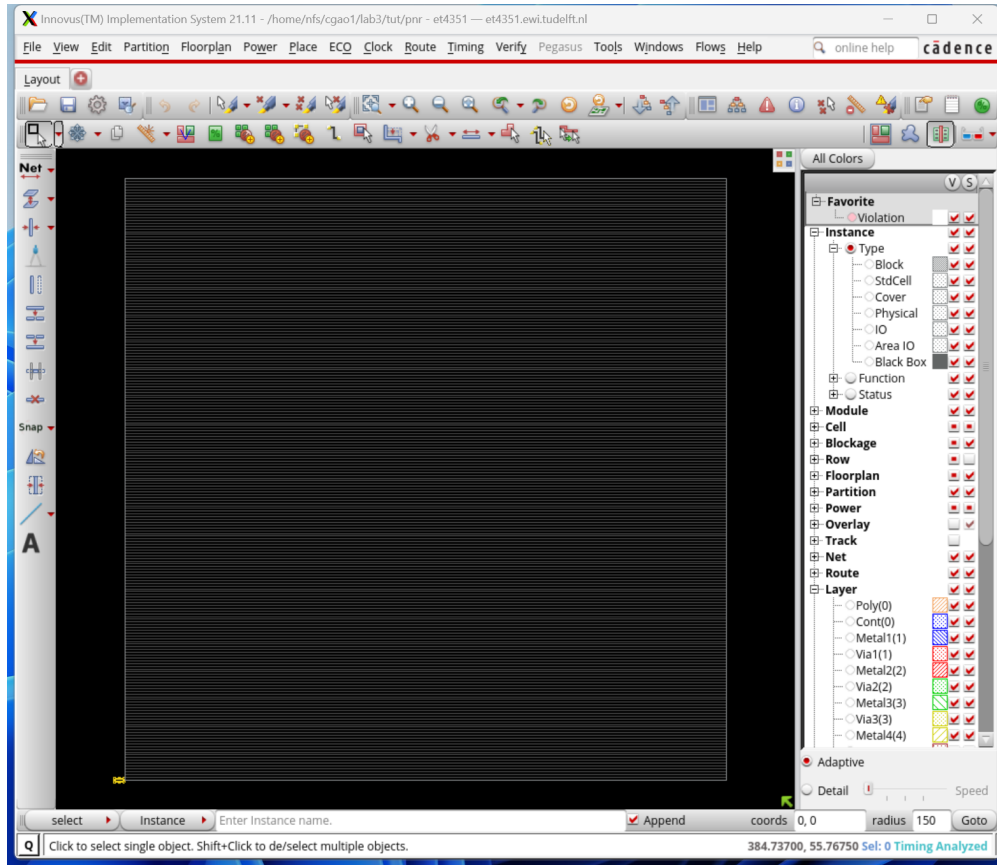


Figure 1: Cadence Innovus GUI with die area defined.

## 3.2 Floorplanning

Execute the script provided below for floorplanning, and while it's running, take the time to peruse the accompanying text. This will help you better understand the script's operations and functions.

---

```
1 source ./scripts/3.0.fplan.tcl
```

---

### 3.2.1 Die Size & Standard Cell Area

Floorplanning begins with defining the size of the die and the area designated for standard cells. The size of the die (chip) and the standard cell area (core) are generally dictated by the complexity of the design and the technology node you are using.

Defining the die size and standard cell area is to provide a high-level layout of the design's physical implementation. This layout determines the total area for placing the standard cells and macros. Furthermore, it guides routing and placement stages later in the design process.

This is done by the following part of the script:

---

```
1 # #####
2 # FLOOR PLAN VARIABLES
3 # #####
4
5 set rowgap 0.0
6
7 # Gap between the core and the power rails (FEED1 size: 0.2 BY 1.71)
8 set coregap 34.2
9
10 # Core dimensions (if first floorPlan option selected below)
11 set Wcore 596.4
12 set Hcore 596.4
13
14 # #####
15 # FLOOR PLAN DRAW
16 # #####
17
18 setFPlanRowSpacingAndType $rowgap 1
19
20 floorPlan -site CoreSite -s $Wcore $Hcore $coregap $coregap $coregap $coregap -
    noSnapToGrid
21
22 loadIoFile ./scripts/3.1.ET4351_chip.io
```

---

This script defines variables and running commands related to floorplanning in a physical design flow, likely using a tool like Cadence Innovus. Let's break it down:

#### 1. Floor Plan Variables

- **rowgap**: This is the gap between rows of standard cells. It's set to 0.0, meaning no additional gap between rows exists.
- **coregap**: The gap between the core of the design and the power rails for accommodating the power rails around the core.
- **Wcore** and **Hcore**: The width and height of the core.

#### 2. Floor Plan Draw

- `setFPlanRowSpacingAndType` Specifies the standard row spacing values and row pattern type. The following argument specifies the row spacing type. Specify 1 for every row or 2 for every other row to apply the spacing value.
- `floorPlan` creates the floor plan. It uses the variables defined earlier to set the size and core gap. The `-s` option specifies the core size and the spacing, in micrometers, between the core edge, which is the margin between the outside edge of the core (head) box. The format is `-s {W H Left Bottom Right Top}`. The `-noSnapToGrid` option means that the rows of standard cells won't be forced to align with a grid, which can give you more flexibility in cell placement.
- `loadIoFile` loads an I/O file defining the locations and types of I/O pads in the design. This file is typically provided by the IC designer or generated by a script.

### 3.2.2 Placing SRAM Macros

Once you have defined the die size and the area for standard cells, the next step is to place the macros. Macros are larger blocks like memory (SRAM, ROM), analog components or complex digital functions pre-designed and used as a single unit in your design. Placing macros is critical because their placement can significantly impact performance, power, and area (PPA).

#### Using SRAM in ASIC Design

In ASIC design, when it comes to the need for large memory units, you typically have two options: a register file or Static Random Access Memory (SRAM). The choice between these two largely depends on your specific needs for memory size, power consumption, and speed. If you decide to use SRAM, there's an important factor to understand: SRAM cannot be synthesized directly from Register-Transfer-Level (RTL) code like other parts of your design. Here's why:

A typical Static Random Access Memory (SRAM) consists of 6 transistors (6T), which is much denser and more power-efficient than the flip-flops used if you try to synthesize memory from RTL code. The exact layout of these cells and the peripheral circuitry required to access them (decoders, sense amplifiers, etc.) highly depend on the specific manufacturing process used. Therefore, generic RTL code cannot define SRAM in a way that optimizes its use for a specific process.

Instead, SRAM is typically provided as a pre-compiled block, or "macro," by the foundry that will manufacture the ASIC. These blocks are designed to optimize their performance and power consumption for the foundry's specific manufacturing process. You can specify the size and configuration of the SRAM (such as the number and width of each word), and the memory compiler (not available for our process) will generate an optimized SRAM block for you.

For Lab 3 and our project, we have provided you with a library of already compiled SRAM macros located in `/data/Cadence/gpdk045_v60/Synopsys_sram`.

The following script places the four **SRAM1RW256x8** macros (declared in `picosoc.v`) that make the 1 kB PicoSoc memory addressed from 0x00000000 to 0x000003FF:

```
1 set sram_gap 2.000
2 set sram_ring 22.000
```

```

3      set sram_w      36.835
4      set sram_h      234.390
5
6      placeInstance soc/memory/sram_3      [expr {$coregap          }] [expr {
          $coregap+$Hcore-$sram_h}] R0
7      placeInstance soc/memory/sram_2      [expr {$coregap+ ($sram_w+$sram_gap)}] [expr {
          $coregap+$Hcore-$sram_h}] R0
8      placeInstance soc/memory/sram_1      [expr {$coregap+2*($sram_w+$sram_gap)}] [expr {
          $coregap+$Hcore-$sram_h}] R0
9      placeInstance soc/memory/sram_0      [expr {$coregap+3*($sram_w+$sram_gap)}] [expr {
          $coregap+$Hcore-$sram_h}] R0
10
11     addHaloToBlock 0  $sram_gap  $sram_gap  0 -fromInstBox soc/memory/sram_3
12     addHaloToBlock 0  $sram_gap  $sram_gap  0 -fromInstBox soc/memory/sram_2
13     addHaloToBlock 0  $sram_gap  $sram_gap  0 -fromInstBox soc/memory/sram_1
14     addHaloToBlock 0  $sram_gap  $sram_gap  0 -fromInstBox soc/memory/sram_0

```

`sram_gap` denotes the distance in micrometers that separates each SRAM instance. In contrast, `sram_ring` represents the dimensions of the halo to be added around the final SRAM instance. This surrounding **halo** serves several essential purposes in chip design:

- **Design Rule Compliance:** A halo ensures that design rules, which dictate the minimum spacing between different layout objects, are not violated, thus contributing to reliable chip operation.
- **Electrical Isolation:** By reducing electrical interference between the SRAM and other circuit components, a halo can help maintain the integrity of the SRAM's operations. This isolation is crucial when digital signal switching in adjacent logic might generate noise that could adversely affect the SRAM.
- **Thermal Management:** In scenarios where an SRAM block produces considerable heat, a halo can help prevent this heat from impacting nearby circuit elements.
- **Congestion Management:** The space within the halo can be utilized for routing wires that connect different parts of the chip, thus helping alleviate routing congestion—a critical aspect of chip design that we've emphasized in our lectures.
- **Ease of Modifications:** Having extra space around the SRAM facilitates potential modifications to the SRAM or surrounding circuitry without necessitating the movement of other design elements.

Finally, the variables `sram_w` and `sram_h` denote the width and height of the SRAM macro, respectively. These dimensions are defined in the `saed32sram.lef` file, specifically at **line 79435**.

In Cadence Innovus, the origin point of the floor plan coordinate system, by default, is the **bottom-left corner (0,0)**. The `placeInstance` commands place four instances of the SRAM on the chip by using three arguments: `x-coordinate`, `y-coordinate` and `orientation`.

- The `expr` command evaluates an expression and returns its value. The x-coordinate takes into account the gap from the core to the SRAM instance `$coregap`, the width of the SRAM `$sram_w`, and the gap between each SRAM `$sram_gap`. The value of n varies from 0 to 3 and represents the number of the SRAM instance.

- The `y-coordinate` of placement is calculated by subtracting the height of the SRAM from the sum of the core height and the gap from the core.
- The `R0` at the end of each `placeInstance` command indicates that the SRAM should be placed with a rotation of 0 degrees.

Finally, the `addHaloToBlock` commands add a **halo** around each SRAM instance with arguments left, bottom, right, and top halo sizes<sup>2</sup>, respectively. The halo for the last SRAM instance (soc/memory/sram\_0) is different, having a larger right-side halo `$sram_ring` than the other instances `$sram_gap`. The `-fromInstBox` option specifies that the halo should be added based on the instance's bounding box.

Remember, floorplanning is an iterative process. You may need to revise your floor plan as you progress through the design process and learn more about your design's specific requirements and constraints.

After placing the SRAM macro instances, your layout should look like Figure 2

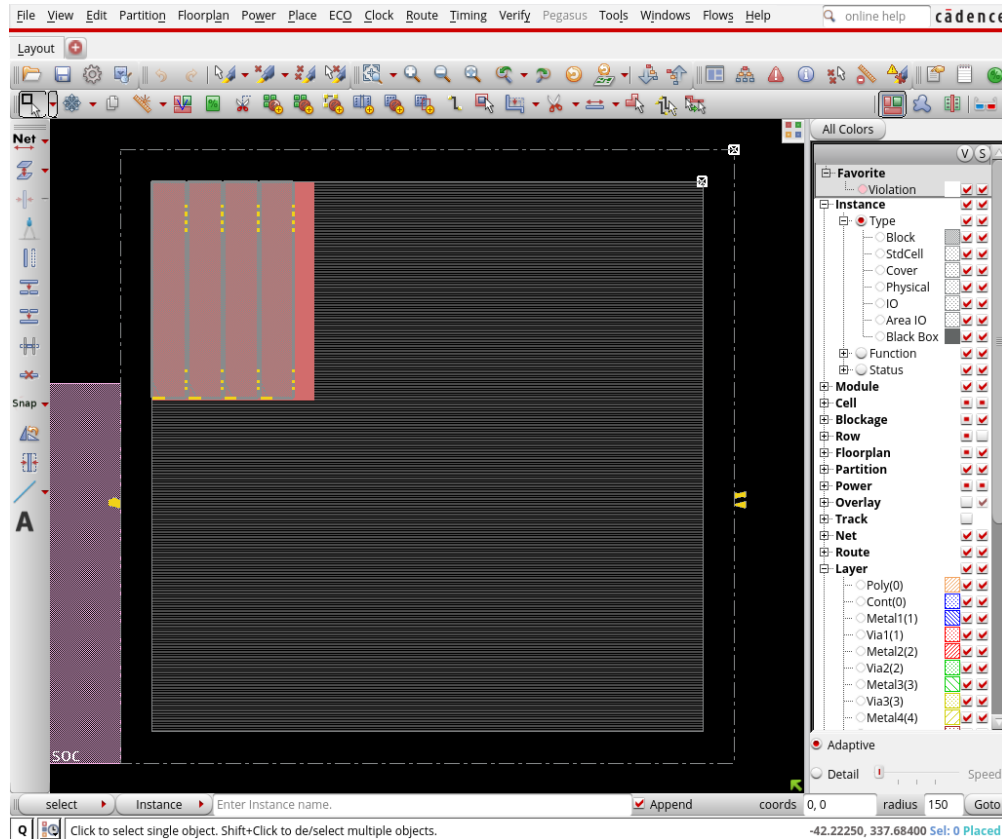


Figure 2: Layout after placing the SRAM macro instances.

### 3.3 Power Planning

Power planning is required to ensure the robust and reliable operation of the chip under various conditions. This tutorial will guide you through the primary stages of power planning for a standard single-power-domain design. The necessary commands are provided in the `4.pplan.tcl` script,

<sup>2</sup>The width of the halo along the left/bottom/right/top edge of instances having halos added.

but we highly recommend executing them one by one manually (copy each line from the script and paste it onto the Innovus command line) while closely observing the actions taking place in the Cadence Innovus GUI.

### 3.3.1 Add Power Rings

The first step is to add power rings surrounding the chip to provide a low-resistance path for the current. They are usually connected directly to the power pads. The width and number of rings are decided based on the total current requirement of the chip and IR drop specifications. Now, run the following commands:

---

```

1  # Chip
2  addRing -around default_power_domain -nets {VSS VDD} -layer {bottom M1 top M1 right
    M2 left M2} -width 12 -spacing 5 -offset 2
3  addRing -around default_power_domain -nets {VSS VDD} -layer {bottom M3 top M3 right
    M4 left M4} -width 12 -spacing 5 -offset 2

```

---

These commands add power rings around the default power domain (the core area). The rings are on metal layers M1/M2 and M3/M4 for power **VDD** and ground **VSS**, with a width of  $12\mu\text{m}$ , a spacing of  $5\mu\text{m}$ , and an offset of  $2\mu\text{m}$  from the boundary of the power domain.

To make the routing of SRAM pins easier, add a vertical stripe using the following command:

---

```

1  # Stripe after macros to ease followpin routing
2  addStripe -number_of_sets 1 -start_from left -start [expr {$coregap+3*($sram_w+
    $sram_gap)+$sram_w+$sram_gap+1}] -spacing 2.0 -direction vertical -layer 6 \
    -width 8 -nets {VSS VDD}

```

---

This command adds a vertical power stripe on metal layer M6 for power **VDD** and ground **VSS**, starting from a calculated position based on the dimensions and placement of the SRAM macros. The stripe is  $8\mu\text{m}$  wide and spaced  $2\mu\text{m}$  apart.

### 3.3.2 Power Connections

The commands below define the global connection between the global power **VDD**, ground **VSS** nets to the power/ground pins **pgpin** of all instances in the design:

---

```

1  globalNetConnect VDD -type pgpin -pin VDD -inst *
2  globalNetConnect VSS -type pgpin -pin VSS -inst *

```

---

Next, define the global connection between **VDD**, **VSS** and all **tie-high** and **tie-low** cells in the design, respectively:

---

```

1  globalNetConnect VDD -type tiehi
2  globalNetConnect VSS -type tielo

```

---

Tie-high and tie-low cells are special standard cells used to connect to the power supply or ground, respectively, to provide a constant logic level. Here we only define the global connection relationship between these cells and VDD, VSS, we will only place and route these cells in the later placement stage.

Macros or IP blocks like memory, processors, etc., have a high power consumption. They usually have their power pins to which power rings and stripes must be connected properly to ensure a robust supply. Now, connect **VDD**, **VSS** to the power and ground pins of the block using the special routing command **sroute**:



---

```

1  # SRAM power connections
2  sroute -nets { VDD VSS } -connect { blockPin }

```

---

### 3.3.3 Draw Power Stripes

Power stripes are routes that run vertically and horizontally across the design, connecting the power rings to the rest of the chip, helping to distribute power evenly and reduce IR drop. Now, add power stripes on metal layers M7 and M8 for **VDD** and **VSS**.

---

```

1  addStripe -number_of_sets 8 -start_from bottom -start 70 -stop 596 -spacing 2.5 \
    -direction horizontal -layer 7 -width 8 -nets {VSS VDD}
2  addStripe -number_of_sets 2 -start_from left -start 70 -stop 155 -spacing 2.5 \
    -direction vertical -layer 8 -width 8 -nets {VSS VDD}
3  addStripe -number_of_sets 5 -start_from left -start 240 -stop 596 -spacing 2.5 \
    -direction vertical -layer 8 -width 8 -nets {VSS VDD}

```

---

Finally, we must connect the **VDD** and **VSS** to the power and ground pins of the core to power standard cells that will be placed during the following placement step:

---

```

1  sroute -nets { VDD VSS } -connect { corePin }

```

---

Now have finished all steps in the floorplanning stage. Your layout should look like Figure 3

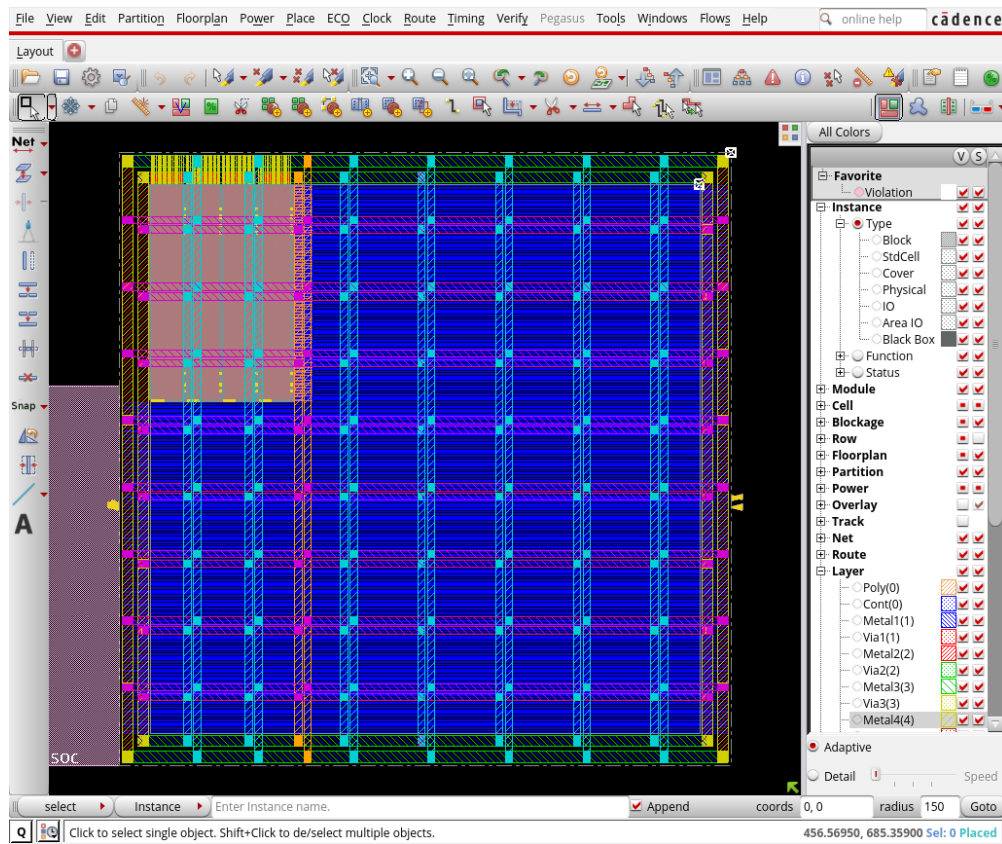


Figure 3: Layout after the floorplanning stage.

### 3.3.4 Some Important Points to Remember

- **IR Drop** refers to the voltage drop that occurs when current (I) flows through resistance (R) in an electrical circuit. As current flows from the power supply to various parts of the chip through metal interconnects, voltage drops will occur due to the inherent resistance of the metal. This can lead to situations where the actual voltage supplied to certain parts of the chip is less than intended.
- **Electromigration** is a phenomenon where metal atoms in the conductor gradually displace due to high current density, increasing resistance and potentially creating open circuits or shorts.
- **Macros** like SRAM often have high current requirements and are often hotspots for IR drop. Therefore, they need a proper supply and should ideally be placed close to supply pads.
- **Decap cells** are special cells with capacitors that are used to reduce the power supply noise. They can be added between the power and ground rails.

## 3.4 Placement

Placement comes after floorplanning and before routing. The goal of placement is to arrange the standard cells in the rows created during floorplanning to optimize for certain objectives:

- **Minimizing total wire length:** This reduces power consumption and signal delay.
- **Timing optimization:** It's important to meet setup and hold time requirements for all paths.
- **Congestion optimization:** Placement should ensure that no region of the design is overly congested to facilitate routing.
- **Power optimization:** Placement can also be used to reduce power consumption.

Run the following script to launch placement and reading the following explanations during the execution:

---

```
1 source ./scripts/5.place.tcl
```

---

### 3.4.1 Activity Annotation

Before proceeding with the placement of standard cells, we must incorporate the switching activity of our design using the `.vcd` file obtained from the post-synthesis simulation. Performing propagation of the switching activities throughout the design before the placement stage enables **Innovus** to yield more precise power estimation results in each subsequent step.

The commands below are used to input the `.vcd` file and propagate the switching activities:

---

```
1 # Give the activity extracted from structural simulations
2 set_power_analysis_mode -report_missing_nets true -corner delay_typ \
   -analysis_view analysis_view_power
3 read_activity_file ../sim_struct/vcd/${DESIGN}.struct.vcd -reset -format VCD \
   -scope testbench/dut
4 propagate_activity
5
6 report_power -outfile powerReports/prePlace_VCDImport.rpt
```

---

- The `set_power_analysis_mode` command sets up the power analysis mode.
  - The `-report_missing_nets true` option ensures that the tool reports any nets in the design that are missing activity data.
  - The `-corner_delay_typ` option sets the typical delay corner for the analysis.
  - The `-analysis_view analysis_view_power` option specifies the analysis view to be used for power analysis.
- The `read_activity_file` command reads the activity `.vcd` file generated from structural simulations.
  - The `-reset` option clears any existing activity data before reading the new file.
  - The `-scope tb_et4351/dut` option specifies the scope for applying the activity data, which is defined in the test bench `tb_et4351.sv`.
- The `propagate_activity` command propagates the activity data through the design hierarchy. This is done to ensure that the activity data is applied to all relevant nets in the design.
- The `report_power` command generates a power report using the current activity data. The report is written to the specified file.

This pre-placement power report is useful to guide the placement stage. For example, cells that consume a lot of power might need to be placed near the power source to minimize IR drop. Moreover, the power consumption data could also influence the decision on whether to use timing-driven or power-driven placement.

### 3.4.2 Place Cells

The following commands place the standard cells in the rows:

---

```

1  setPlaceMode -place_global_cong_effort high \
2      -place_global_solver_effort high \
3      -place_global_timing_effort medium \
4      -place_global_activity_power_driven true \
5      -place_global_activity_power_driven_effort standard \
6      -place_global_uniform_density true
7  setDesignMode -topRoutingLayer 10
8  set delaycal_use_default_delay_limit 1000
9  setExtractRCMode -engine preRoute -effortLevel high
10
11 placeDesign
12 place_opt_design -incremental -out_dir timingReports/place_opt_design.rpt
13
14 setDrawView place
15
16 trialRoute -noDetour

```

---

- The `setPlaceMode` command controls the effort level of the global placement process. The parameters suggest a high effort on congestion and solving the placement, a medium effort on timing, and a standard effort on power, with power-driven activity being enabled since we provided the `.vcd` file for activity annotation. Finally, uniform cell distribution is enabled for designs with less than 70% core area utilization (density).

- The `setDesignMode -topRoutingLayer` command sets the top routing layer to be M10.
- The `set delaycal_use_default_delay_limit 1000` command sets the pin number threshold for nets for delay calculation. Nets with more than the specified number of pins are excluded from delay calculation and use the default delay values instead. This is particularly useful for large designs.
- The `setExtractRCMode -engine preRoute -effortLevel high` command sets the RC extraction mode to `preRoute` and the effort level to `high`. RC extraction is the process of calculating the parasitic resistance and capacitance values in the design. `preRoute` means the RC extraction is performed before routing and after Clock Tree Synthesis (CTS) by the fast density measurements of the surrounding wires; coupling is not reported.
- The `placeDesign` command runs the placement process on the design. This is where the tool places the cells in the design onto the core area.
- The `place_opt_design` command runs placement optimization on the design incrementally and writes the report to the specified directory. Placement optimization adjusts the locations of the cells to further improve timing, reduce congestion and optimize power after initial placement.
- The `setDrawView place` command sets the view to the placement view, which allows us to visually inspect the placement of the cells in the design.
- The `trialRoute -noDetour` command runs a trial route on the design without any detour<sup>3</sup>. The trial route is a quick estimate of the routing process to get an idea of the wire length, congestion, and timing.

The following command adds the Tie High and Tie Low cells:

---

```
1 addTieHiLo -prefix TIEHILO_ -cell "TIEHI TIELO"
```

---

The layout after placement is shown in Figure 4.

### 3.4.3 Generate Reports

To evaluate the design's performance throughout the physical implementation process, we aim to generate timing and power reports for each step: pre-cts, post-cts, post-route, and final. The provided commands below serve as an example for generating reports during the pre-cts stage:

---

```
1 set_power_analysis_mode -method static -corner delay_typ -create_binary_db true -
  write_static_currents true -honor_negative_energy true -ignore_control_signals
  true -analysis_view analysis_view_power
2
3 timeDesign -preCTS -outDir timingReports/preCTS
4 timeDesign -preCTS -hold -outDir timingReports/preCTS_hold
5 report_power -outfile powerReports/preCTS.rpt
6
```

---

<sup>3</sup>A "detour" in routing refers to the situation where a routing path needs to divert from the most direct or shortest path between two points (say, between two pins of different cells) due to obstacles or congestion. Obstacles might include pre-placed cells, blocks, or areas of the design that are already highly congested with other routing paths. When a detour is needed, the router must find an alternate, typically longer path, which could potentially increase the delay and power consumption.

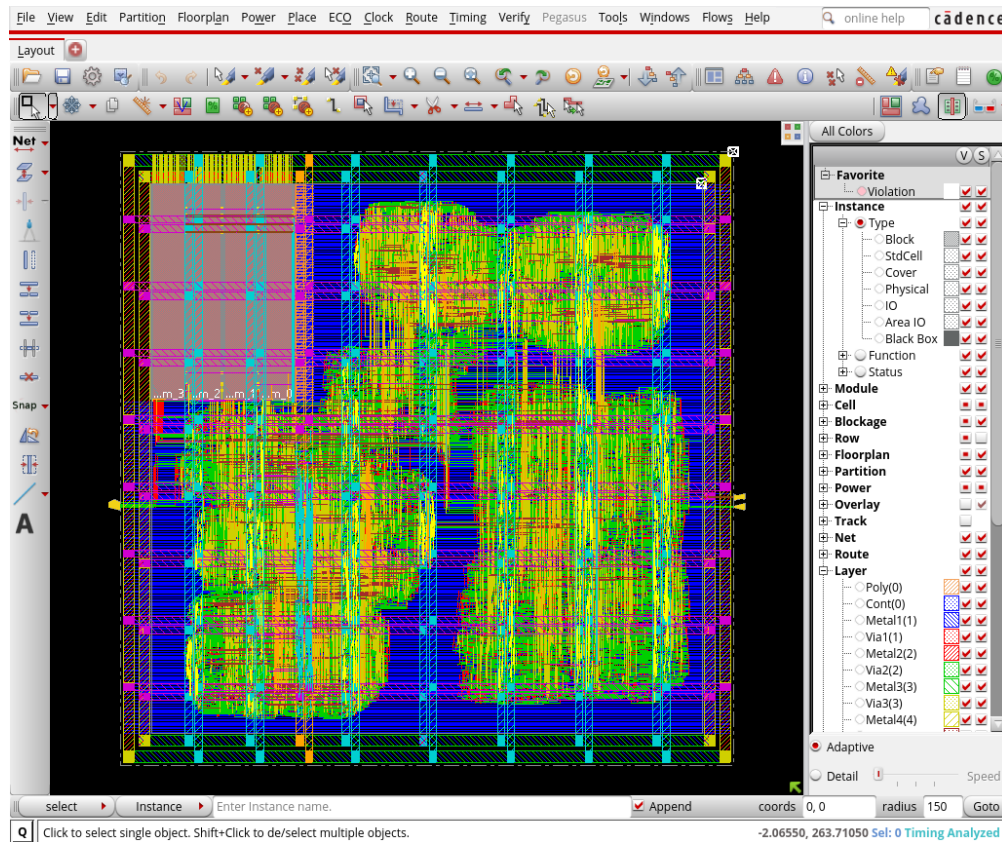


Figure 4: Layout after the placement stage.

```
7 reportDensityMap > densityReports/density.rpt
8 checkPlace verifyReports/checkPlace.rpt
```

- The `set_power_analysis_mode` command sets the power analysis mode for the design.
- The `timeDesign -preCTS -outDir timingReports/preCTS` command runs timing analysis on the design before CTS is performed and stores the results in a specific directory.
- The `timeDesign -preCTS -hold -outDir timingReports/preCTS_hold` command runs hold timing analysis.
- The `report_power -outfile powerReports/preCTS.rpt` generates a power report for the current state of the design.

### 3.5 Clock Tree Synthesis (CTS)

CTS is a pivotal process to distribute the clock signal from its source to all endpoint pins, including registers, clock gates, and macro clock pins. The clock tree significantly influences the chip's performance, power, and area metrics and ensures synchronization across the chip.

Run CTS by:

```
1 source ./scripts/6.cts.tcl
```

CTS is done by the following commands:

```
1  setOptMode -usefulSkew true
2  setOptMode -usefulSkewCCOpt extreme
3  set_ccopt_property buffer_cells {          CLKBUF2 CLKBUF3 CLKBUF4 CLKBUF6
      CLKBUF8 CLKBUF12 CLKBUF16 CLKBUF20}
4  set_ccopt_property inverter_cells {CLKINVX1 CLKINVX2 CLKINVX3 CLKINVX4 CLKINVX6
      CLKINVX8 CLKINVX12 CLKINVX16 CLKINVX20}
5  set_ccopt_property delay_cells {DLY1X1 DLY1X4 DLY2X1 DLY2X4 DLY3X1 DLY3X4
      DLY4X1 DLY4X4}
6
7  ccopt_design
```

- `setOptMode -usefulSkew true` allows useful skew during optimization. In certain extensive designs, striving for absolute zero skews can incur unnecessary expenses. When clock skew is deliberately utilized to satisfy timing requirements, it is referred to as useful skew.
- `setOptMode -usefulSkewCCOpt extreme` sets the optimization effort level to 'extreme' for useful skew.
- `set_ccopt_property buffer_cells ...` specifies the cells that can be used as buffers in the clock tree.
- `set_ccopt_property inverter_cells ...` specifies the cells that can be used as inverters in the clock tree.
- `set_ccopt_property delay_cells ...` specifies the cells that can be used as delay cells in the clock tree.
- `ccopt_design` launches CTS on the design.

The CTS is followed by post-CTS optimization:

```
1  setOptMode -fixCap true -fixTran true -fixFanoutLoad true
2  setOptMode -holdTargetSlack 0.1
3  optDesign -postCTS -hold -outDir timingReports/postCTSHold_hold
4  timeDesign -postCTS -outDir timingReports/postCTSHold
```

- `setOptMode -fixCap true -fixTran true -fixFanoutLoad true` sets the optimization mode to fix capacitive load, transition times, and fanout load issues.
- `setOptMode -holdTargetSlack 0.005` sets our hold slack target to be 0.005. The larger the value, the more effort the tool will spend on optimizing for hold timing closure.
- `optDesign -postCTS -hold -outDir timingReports/postCTSHold_hold` runs optimization on the design after CTS to fix hold violations, and it outputs the timing reports to the specified directory.

The layout after clock tree synthesis should look like Figure 5



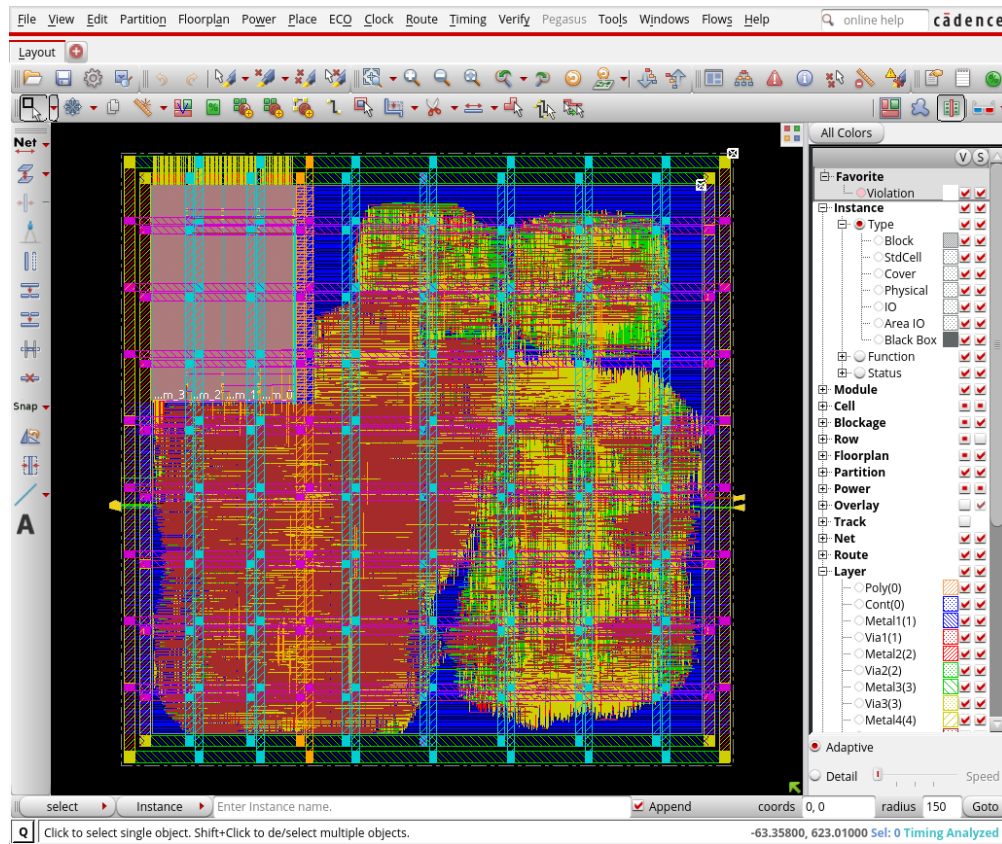


Figure 5: Layout after the clock tree synthesis stage.

## 3.6 Routing

In most designs, metal layers have preferred directions - odd layers run horizontally, and even layers run vertically. The goal of the routing stage is to route all signal nets between macros, IOs, and standard cells, ensuring a violation-free design.

Run the routing stage by:

```
1 source ./scripts/7.route.tcl
```

### 3.6.1 Filler Cells

Filler cells fill the gaps to maintain a uniform density and ensure the continuity of wells and power lines. By filling empty spaces, they help maintain a consistent poly density across the design, contributing to better manufacturability and reliability. The following commands added the filler cells:

```
1 # Filler configuration and inclusion
2 setFillerMode -core {FILL1 FILL2 FILL4 FILL8 FILL16 FILL32 FILL64} -corePrefix
   FILLER_ -doDRC true -ecoMode true
3 addFiller
```

### 3.6.2 Nano Route Parameters

The `setNanoRouteMode` command is used to configure various parameters for the routing process:

```
1  setNanoRouteMode -quiet -routeTdrEffort 5
2  setNanoRouteMode -quiet -routeBottomRoutingLayer default
3  setNanoRouteMode -quiet -drouteEndIteration 30
4  setNanoRouteMode -quiet -routeWithTimingDriven true
5  setNanoRouteMode -quiet -routeWithSiDriven true
6  setNanoRouteMode -quiet -routeSiEffort high
7  setNanoRouteMode -drouteFixAntenna true
8  setNanoRouteMode -routeFixTopLayerAntenna false
9  setNanoRouteMode -routeInsertAntennaDiode true
10 setNanoRouteMode -routeAntennaCellName "ANTENNA"
11 setOptMode      -postRouteDrvRecovery true -fixSISlew true -fixGlitch true
```

Here are explanations of some of the previous options:

- `-routeTdrEffort 5` sets the level of effort for timing-driven routing. Higher values mean more effort, which usually translates to better results but longer runtime.
- `-routeBottomRoutingLayer default` and `-drouteEndIteration 30` specify the starting routing layer and the number of iterations for detailed routing.
- `-routeWithSiDriven true` and `-routeSiEffort high` enable and set the effort level for signal integrity-driven routing.
- The final `setOptMode` configures the optimization mode after routing.

### 3.6.3 RC Extraction

The command `setExtractRCMode -engine postRoute -effortLevel high` sets the engine for resistance-capacitance (RC) extraction to `postRoute` and the effort level to `high`.

### 3.6.4 Routing Execution

The `routeDesign` command performs the actual routing operation:

```
1  routeDesign -globalDetail
2  ecoRoute -fix_drc
```

The `ecoRoute -fix_drc` command is used to perform Engineering Change Order (ECO) routing. ECOs are changes made to the design after some level of completion has been achieved (after the design has been frozen). The changes could be to fix design errors, implement new requirements, or optimize the design. The `ecoRoute` command specifically pertains to making routing changes to the design.

The `fix_drc` option instructs the tool to fix Design Rule Check (DRC) violations during the ECO routing process. DRC is a step in the IC design cycle that checks whether the design obeys a set of recommended parameters (rules) the semiconductor foundry defines. DRC violations occur when these rules are not met.

Next, the following script generates timing and power reports after routing and before optimization. These reports are useful for identifying problem areas that need addressing during the optimization phase:



---

```

1  timeDesign -postRoute -pathReports -slackReports -numPaths 50 -outDir timingReports/
   route
2  timeDesign -postRoute -hold -pathReports -slackReports -numPaths 50 -outDir
   timingReports/route_hold
3
4  set_power_analysis_mode -method static -corner delay_typ -create_binary_db true -
   write_static_currents true -honor_negative_energy true -ignore_control_signals
   true -analysis_view analysis_view_power
5
6  report_power -clock_network all -hierarchy all -cell_type all -power_domain all -
   pg_net all -sort { total } -outfile powerReports/route.rpt

```

---

### 3.6.5 Post-Route Optimization

The optimization is performed after routing to improve the design's performance:

---

```

1  setOptMode -fixCap true -fixTran true -fixFanoutLoad true -postRouteHoldRecovery auto
2  optDesign -postRoute

```

---

The following script generates timing and power reports after the optimization:

---

```

1  timeDesign -postRoute -pathReports -slackReports -numPaths 50 -outDir timingReports/
   postRoute
2  timeDesign -postRoute -hold -pathReports -slackReports -numPaths 50 -outDir
   timingReports/postRoute_hold
3
4  set_power_analysis_mode -method static -corner delay_typ -create_binary_db true -
   write_static_currents true -honor_negative_energy true -ignore_control_signals
   true -analysis_view analysis_view_power
5
6  report_power -clock_network all -hierarchy all -cell_type all -power_domain all -
   pg_net all -sort { total } -outfile powerReports/postRoute.rpt

```

---

The following part performs hold time optimization after routing and generates timing, noise, and power reports:

---

```

1  setOptMode -fixCap true -fixTran true -fixFanoutLoad true -postRouteHoldRecovery auto
2  setOptMode -holdTargetSlack 0.02
3  optDesign -postRoute -hold
4  optDesign -postRoute -drv
5
6  timeDesign -postRoute -pathReports -slackReports -numPaths 50 -outDir timingReports/
   postRouteHold
7  timeDesign -postRoute -hold -pathReports -slackReports -numPaths 50 -outDir
   timingReports/postRouteHold_hold
8  report_noise -bumpy_waveform -output_file timingReports/bumpyWaves_postRouteHold.rpt
9
10 set_power_analysis_mode -method static -corner delay_typ -create_binary_db true -
   write_static_currents true -honor_negative_energy true -ignore_control_signals
   true -analysis_view analysis_view_power
11
12 report_power -clock_network all -hierarchy all -cell_type all -power_domain all -
   pg_net all -sort { total } -outfile powerReports/postRouteHold.rpt

```

---

The `optDesign -postRoute -hold` command launches the post-layout optimization for hold violation fixing using tricks such as:

- **Buffer Insertion:** Innovus may add buffers to increase the delay along certain paths that are violating hold time constraints. This helps to ensure that data does not arrive too early at sequential elements.
- **Buffer Removal or Upsizing:** Conversely, if a buffer is causing unnecessary delay on a path, Innovus may choose to remove it or replace it with a larger-sized buffer to decrease the delay.
- **Cell Re-sizing:** Innovus can also resize cells to modify their delay characteristics. The delay can be reduced by making a cell larger (upsizing) or smaller (downsizing) to increase the delay.
- **Wire Re-routing:** In some cases, Innovus might slightly adjust the routing of interconnect wires to add or reduce delay, helping to meet hold time constraints.
- **Gate Cloning:** For complex designs, Innovus can duplicate certain gates to help balance the loads on various paths and address hold time violations.

These optimizations are made while trying to maintain the design's overall timing, power, and area characteristics, and they should not introduce new setup time violations. The optimizations might lead to an increase of cells and connections, leading to higher density, as shown in Figure 6

### 3.7 Verification & Signoff

After the design is placed and routed, it's important to check and verify that it meets all design rules and timing requirements.

Run verification and all remaining steps by:

---

```
1 # Make sure that you have 0 DRC violations and 0 antenna violations!
2 source ./scripts/8.verify.tcl
3 source ./scripts/9.report.tcl
4 source ./scripts/10.export.tcl
```

---

**Now, make sure that you keep Innovus active in your current terminal and run the remaining steps of this document in a new/separate terminal!!!**

The reports are generated in the folders named **initialReports**, **clockReports**, **timingReports**, **powerReports**, **verifyReports** and **finalReports** under the `~/lab3/tut/pnr` directory. Some reports are stored in `gzip` compressed packs and can be decompressed using the following command:

---

```
1 gzip -d <filename>.gz
```

---

Here we mainly explain two reports:

#### 3.7.1 Setup and Hold Timing Reports

Setup and hold timing reports are essential for ensuring that your design meets its performance objectives.

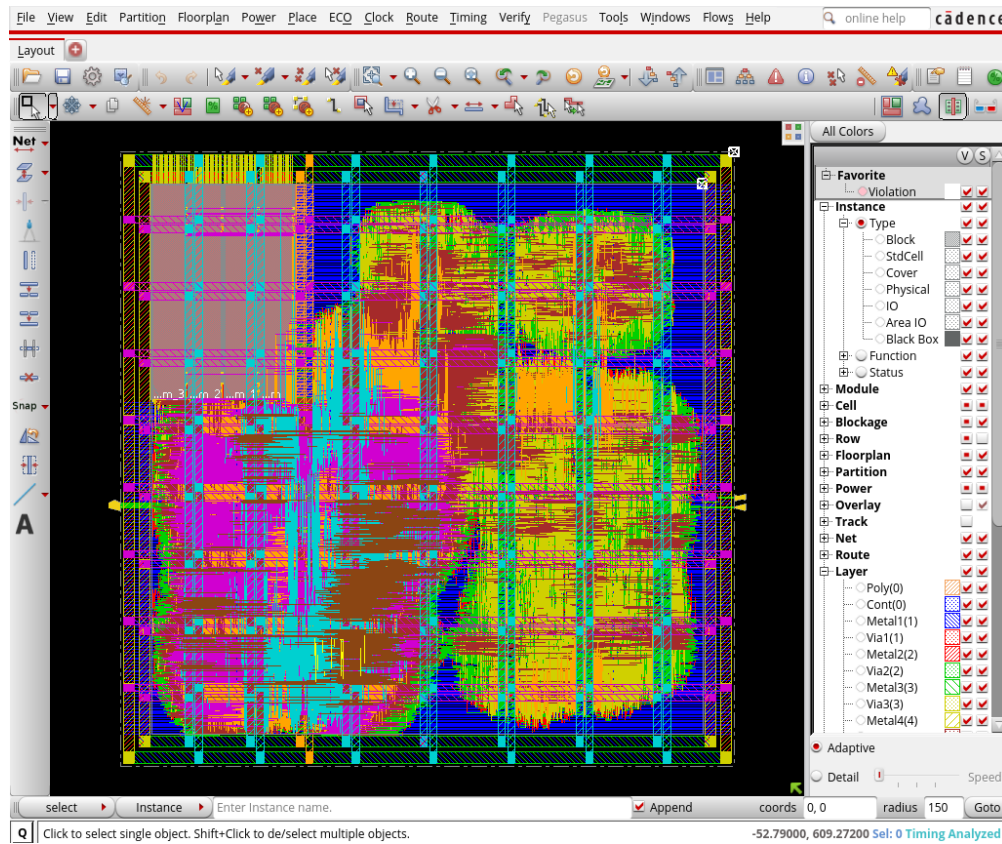


Figure 6: Layout after the routing stage.

- **Setup Timing Reports:** Check these reports for the slowest propagation time. Setup reports help to identify any setup violations in your design, which occur when data arrives too late for a flip-flop to latch it correctly.
- **Hold Timing Reports:** Hold timing reports should be checked for the fastest propagation time. Hold violations occur when data changes too quickly after the clock edge, causing the flip-flop to latch incorrect data.

### 3.7.2 Design Rule Violation (DRV) Reports

Design Rule Violation (DRV) reports are crucial in ensuring your design adheres to the rules stipulated by the technology process. Some key DRV reports that warrant close attention include the following:

- **Max Cap** flags any cells exceeding the maximum allowable load capacitance, which could lead to driving strength and timing issues. This is often considered the most precise measure and should be given priority.
- **Max Transition** identifies nets where the signal transition time exceeds the maximum allowable limit, potentially causing timing issues. Signal transition time refers to the duration it takes for a signal to switch from a low to a high state or vice versa.

- **Max Fanout** details the maximum number of gates that a single output pin can drive. It is used mainly for early estimates with the wire load model and is generally considered less crucial than the Max Cap report.
- **Max Length** indicates the maximum physical length a net can have.

While normally, max pulse width DRVs are part of these reports, they may not always appear directly in the DRV reports. Instead, any such violations may show up when using the `report_noise` command, of which the reports should also be clean and free of violations.

You might find some Max Tran violations in the DRV report due to the multi-driven nets in the PicoSoC. You may ignore them for now, as we will optimize their source code further for your project.

### 3.8 Post-Layout Simulation

The post-layout simulation should be done twice to verify both the min and max corners. In digital design, **setup** and **hold** time are the two important parameters that define the validity of input signals to a flip-flop. These constraints ensure that the data is correctly captured and processed.

The reason we use the maximum delay corner (also known as "worst-case" or "slow") for setup analysis and the minimum delay corner (also known as "best-case" or "fast") for hold analysis is due to the nature of these constraints:

- **Setup Analysis:** The setup time is the minimum amount of time the data should be held steady before the clock event so the clock edge reliably samples the data. Here, we are concerned about the data not reaching the flip-flop before the clock edge. Therefore, we consider the worst-case scenario when the data path delay is at its maximum, typically at the minimum specified voltage and maximum temperature. In this case, the data might take the longest time to reach the flip-flop, causing a setup violation if the clock edge comes before the data stabilizes. Hence, we use the 'max' corner to ensure the design is safe under these conditions.
- **Hold Analysis:** The hold time, conversely, is the minimum amount of time the data should be held steady after the clock edge. In this case, we are concerned about the data changing too quickly after the clock edge, which could lead to incorrect data sampling. Thus, we consider the best-case scenario, where the data path delay is minimal; typically, when the supply voltage is at its maximum specified level, the temperature is at its lowest. This means the data could change quickly after the clock edge, causing a hold violation. Using the 'min' corner ensures the design is safe under these conditions.

By performing these analyses, designers can ensure that digital circuits function correctly under various operating conditions and manufacturing variations.

As the post-layout simulation generates a very large VCD file when the activity from all clock cycles from start to finish are stored account, we propose an alternative approach here. First, run the behavioral simulation of the design:

---

```
1 cd ~/lab3/tut/sim_behav && source run_behav_sim.sh
```

---

At the bottom of the output, you should see the following, indicating how much time (cycles) the complete execution took, as well as the starting time of the accelerated sorting algorithm and its duration:

---

```
1 # Complete latency in Clock Cycles:      2600653
2 # Accelerator start time in Clock Cycles:    400825
3 # Accelerator runtime in Clock Cycles:      1887
4 # Complete latency in Milliseconds:      216.712414
5 # Accelerator start time in Milliseconds:    33.400747
6 # Accelerator runtime in Milliseconds:      0.157244
```

---

Now, we will use this data to only store the waveforms during the period of accelerator execution, not just making the VCD file smaller but also making the power estimation more accurate by measuring the power in a window where our accelerator is constantly running.

To do this, edit both `sim_phys/scripts/run_vcd_hold.cmd` and `sim_phys/scripts/run_vcd_setup.cmd`, where you should update line 7 and line 17 with the accelerator start time (33.400747 above) and accelerator runtime (0.157244) in ms respectively.

Now, run the post-layout simulation for setup analysis:

---

```
1 cd ../sim_phys
2 mkdir vcd
3 source run_pnr_sim_setup_max.sh
```

---

Then run the post-layout simulation for hold analysis:

---

```
1 source run_pnr_sim_hold_min.sh
```

---

Your physical design & post-layout simulation are successful once you see the correctly sorted numbers printed on your terminal. To verify this, run:

---

```
1 python verify.py
```

---

If everything is correct, the output of this command should be Test Passed ^\_^ Outputs and Gold are identical!!!.

With both simulation scripts having generated `.vcd` files in the `/lab3/tut/sim_phys/vcd` folder, they can now be used for power estimation. Go back to your terminal with Innovus active and feed back the switching activities to the layout to extract the most accurate power numbers:

---

```
1 set_power_analysis_mode -report_missing_nets true -corner delay_min -analysis_view
  analysis_view_power
2 read_activity_file ../sim_phys/vcd/${DESIGN}.phys.hold.vcd -reset -format VCD -scope
  testbench/dut
3 propagate_activity
4
5 report_power -outfile powerReports/Final_min_VCDImport.rpt
```

---

Note: double-check that the coverage is 100%!

## 4 Questions

For Lab 3, please provide individual answers to the following questions, as they may be related to potential inquiries during your oral examination. If you don't like the word template, you are free to use your own tool of choice (e.g., LaTeX or whatever) to edit your report, but please remain roughly the same font size and overall length. You can put figures near your answers. An appendix is not compulsory.

1. Please re-execute the synthesis process for PicoSoC while enabling clock gating, as instructed in Lab 2. Next, proceed with the physical design process and generate a report containing the power estimation results after the layout. Finally, compare these post-layout power estimation figures with the results obtained without clock gating.

Put your post-layout final<sup>4</sup> power estimation report table of PicoSoc in your main body text.

2. Have you observed that the activity annotation coverage rate of the nets decreases in incremental steps following placement? Can you speculate on possible actions that Innovus might have taken with the design that could explain this behavior?
3. Compare the power numbers derived from post-layout activity annotation with those obtained from post-synthesis activity annotation (propagate the two `.vcd` files through your final design that has gone through after all the physical implementation stages). Provide a brief explanation for discrepancies observed between these two sets of numbers.

## 5 Feedback

We greatly value your feedback and would appreciate it very much if you could spare a few minutes to share your thoughts about the labs using the following link or scanning the QR code. Your insights will be crucial in our ongoing efforts to enhance them for future sessions.



Figure 7: Click Here to Give Feedback.

---

<sup>4</sup>After all the physical implementation steps.